Cryptography in Python

Understanding encryption, types, and libraries

What is Cryptography in Python?

- Cryptography is the practice of securing communication and data through encoding.

- In Python, various libraries allow developers to implement encryption, decryption, hashing, and key management.

Plain Text vs Cipher Text

- Plain Text: Original, readable message
- Cipher Text: Encrypted, unreadable version of the message

- Encryption converts plain text to cipher text, and decryption reverses it.

Types of Cryptography

- Symmetric Encryption: Same key used for encryption and decryption (e.g., Fernet)
- Asymmetric Encryption: Uses public and private key pairs (e.g., RSA)

- Symmetric is faster; asymmetric is more secure for communication.

Symmetric Encryption (Example)

- from cryptography.fernet import Fernet
- key = Fernet.generate_key()
- f = Fernet(key)
- token = f.encrypt(b'Secret message')
- message = f.decrypt(token)

Asymmetric Encryption (Example)

- from cryptography.hazmat.primitives.asymmetric import rsa
- private_key = rsa.generate_private_key(...)
- public_key = private_key.public_key()
- # Encrypt with public_key, decrypt with private_key

Popular Python Cryptography Libraries

- - cryptography
- - PyCryptodome
- - hashlib
- - rsa
- - ssl
- - base64

Real-World Applications

- - Secure messaging (e.g., WhatsApp)
- - Online transactions and digital signatures
- - Password storage (hashing)
- - Secure file storage and sharing